SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

QUESTIONS			Total Marks: 50
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	**5**
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	**5**
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	**5**
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	**5**
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	**5**

6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	**5**
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	**5**
8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	**5**
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	**5**
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	**5**

Code of Conduct:

I B Aishwarya/192421432 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively

Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

Q1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Answer:

Query processing is the process of converting an SQL query into an efficient execution plan and then executing that plan to produce the required result.

Steps of Query Processing

1. SQL Query Input

The user first submits an SQL query.

Example:

```
SELECT Name
FROM Student
WHERE Department = 'IT';
```

2. Parsing and Validation

The DBMS checks:

- SQL syntax
- Table names
- Column names
- Data types
- User permissions

If there is an error, the DBMS returns an error message.

3. Translation into Relational Algebra

The SQL query is converted into an internal representation such as relational algebra.

For example:

```
Selection(Student, Department = 'IT')
```

4. Query Optimization

The DBMS generates different possible ways of executing the query.

For example:

- Full table scan
- Index scan
- Different join orders
- Different join algorithms

5. Cost Estimation

The optimizer estimates the cost of each plan using:

- Number of disk I/O operations
- CPU cost
- Memory usage
- Number of tuples
- Available indexes
- Selectivity of conditions

6. Selection of Best Plan

The optimizer chooses the execution plan with the lowest estimated cost.

7. Query Execution

The selected execution plan is passed to the query execution engine.

8. Result Generation

The execution engine retrieves the required records and returns the result to the user.

How does the optimizer choose the best plan?

The optimizer compares different execution plans.

For example:

Plan 1 → Full Table Scan → Cost = 500

Plan 2 → Index Scan → Cost = 100

Since Plan 2 has a lower cost, the optimizer selects the index scan.

Simple Flow

SQL Query

↓

Parsing

↓

Translation

↓

Query Optimization

↓

Cost Estimation

↓

Best Execution Plan

↓

Execution

↓

Result

Conclusion

Query processing converts an SQL query into an executable form, while query optimization selects an efficient execution plan. This reduces execution time and resource usage.

Q2. What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Answer:

Query optimization is the process of selecting an efficient execution plan for a query.

There are two common approaches:

1. Heuristic optimization
2. Cost-based optimization

1. Heuristic Optimization

Heuristic optimization uses predefined rules to improve a query.

Common rules include:

- Perform selection as early as possible.
- Perform projection as early as possible.
- Reduce intermediate relations.
- Perform smaller joins first.
- Avoid unnecessary operations.

Example

Instead of:

Student JOIN Course

↓

Selection Department='IT'

we can apply selection first:

Selection Department='IT'

↓

Student

↓

This reduces the number of tuples before the join.

Advantages

- Simple
- Fast
- Easy to implement
- Requires less statistical information

Disadvantages

- May not always produce the optimal plan
- Does not accurately consider actual execution cost

2. Cost-Based Optimization

Cost-based optimization generates multiple execution plans and estimates their costs.

The cost may include:

- Disk I/O
- CPU processing
- Memory usage
- Network cost
- Number of records
- Available indexes

The plan with the lowest estimated cost is selected.

Example

Plan A → Cost = 1000

Plan B → Cost = 500

Plan C → Cost = 750

The optimizer selects **Plan B**.

Comparison

Heuristic	**Cost-Based**
Uses predefined rules	Uses estimated costs
Faster	More computation required
Simple	More complex
May not find best plan	Usually finds a better plan
Does not require detailed statistics	Uses database statistics
Good for simple queries	Good for complex queries

When to use?

Heuristic optimization:

- Simple queries
- When quick optimization is needed
- When statistics are unavailable

Cost-based optimization:

- Complex queries
- Multiple joins
- Large databases
- When indexes and statistics are available

Conclusion

Heuristic optimization is faster and simpler, while cost-based optimization is more accurate and suitable for complex database queries.

Q3. How would you implement database connectivity in a real project? Explain JDBC and ODBC.

Answer:

Database connectivity allows an application to communicate with a database to insert, retrieve, update, and delete data.

For example, a hospital application may connect to MySQL to store patient information.

Steps for Database Connectivity

1. Install Database Driver

The appropriate driver is installed depending on the programming language and database.

2. Create Connection

The application provides:

Database URL

Username

Password

3. Create SQL Statement

The application creates a statement or prepared statement.

4. Execute Query

SQL commands such as:

SELECT

INSERT

UPDATE

DELETE

5. Process Result

For a SELECT query, the returned records are processed by the application.

6. Commit or Rollback

If the transaction is successful:

COMMIT

If an error occurs:

ROLLBACK

7. Close Resources

The application closes:

- Result set
- Statement
- Database connection

JDBC

JDBC stands for Java Database Connectivity.

It is an API that allows Java programs to communicate with databases.

Basic flow:

Java Application

↓

JDBC API

↓

JDBC Driver

↓

Database

ODBC

ODBC stands for Open Database Connectivity.

It is a standard interface that allows applications to communicate with different database systems through ODBC drivers.

JDBC vs ODBC

JDBC	**ODBC**
Java-specific API	Language-independent standard
Mainly used by Java applications	Used by applications in various languages
Uses JDBC drivers	Uses ODBC drivers
Designed for Java	Designed for general database access

Example

A Java hospital management application can use JDBC to connect to MySQL:

Hospital Application

↓

JDBC

↓

MySQL Driver

↓

MySQL Database

Conclusion

Database connectivity is essential for real-world applications. JDBC is commonly used for Java applications, while ODBC provides a general database connectivity standard.

Q4. What are ACID properties? Give a real-world example where each property is critical.

Answer:

ACID represents four important properties of database transactions:

- **A – Atomicity**
- **C – Consistency**
- **I – Isolation**
- **D – Durability**

They ensure that database transactions are reliable and correct.

1. Atomicity

Atomicity means a transaction is treated as a **single unit**.

It must either:

- Complete completely, or
- Not happen at all.

Example

Suppose ₹5,000 is transferred from Account A to Account B.

Deduct ₹5,000 from A

↓

Add ₹5,000 to B

If the second operation fails, the first operation must also be cancelled.

Therefore:

Either both operations happen

OR

neither happens

2. Consistency

Consistency means a transaction must take the database from one **valid state to another valid state**.

Example

Before transfer:

A = ₹10,000

B = ₹5,000

Total = ₹15,000

After transferring ₹2,000:

A = ₹8,000

B = ₹7,000

Total = ₹15,000

The total remains consistent.

3. Isolation

Isolation means transactions running simultaneously should not interfere incorrectly with one another.

Example

Two customers try to withdraw money from the same bank account simultaneously.

Isolation ensures that both transactions do not incorrectly use the same old balance.

4. Durability

Durability means once a transaction is committed, its changes remain permanently stored even after a system crash.

Example

After a successful bank transfer:

COMMIT

Even if the server crashes immediately afterward, the updated account balance must remain.

Summary

Property	Meaning	Example
Atomicity	All or nothing	Bank transfer
Consistency	Valid database state	Correct account totals
Isolation	Transactions don't interfere	Simultaneous withdrawal
Durability	Committed data is permanent	Data after system crash

Conclusion

ACID properties are especially important in banking, hospital, railway, e-commerce, and other systems where data correctness is critical.

Q5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Answer:

Concurrency control manages multiple transactions executing at the same time while maintaining database correctness.

Without concurrency control, several problems can occur.

1. Dirty Read

A dirty read occurs when one transaction reads data modified by another transaction that has not committed.

Example:

T1: Update Balance = ₹5000

T2: Read Balance = ₹5000

T1: Rollback

T2 has read a value that was never permanently stored.

2. Lost Update

A lost update occurs when two transactions update the same data and one update overwrites the other.

Example:

Initial balance = ₹10,000

T1 reads ₹10,000

T2 reads ₹10,000

T1 updates → ₹9,000

T2 updates → ₹8,000

T1's update is lost.

3. Unrepeatable Read

An unrepeatable read occurs when a transaction reads the same record twice and gets different values.

Example:

T1 reads Balance = ₹10,000

T2 updates Balance = ₹8,000

T2 commits

T1 reads Balance again = ₹8,000

The same query produced different results.

Two-Phase Locking (2PL)

2PL is a concurrency control protocol that uses locks.

There are two types:

Shared Lock (S)

Used for reading.

Multiple transactions can have shared locks simultaneously.

Exclusive Lock (X)

Used for writing.

Only one transaction can hold an exclusive lock.

Two Phases

Growing Phase

Transaction can acquire locks but cannot release them.

Shrinking Phase

Transaction can release locks but cannot acquire new locks.

Growing Phase → Lock Acquisition

↓

Lock Point

↓

Shrinking Phase → Lock Release

How 2PL helps

Problem	**2PL protection**
Dirty Read	Locks prevent reading inappropriate uncommitted changes
Lost Update	Exclusive locks prevent simultaneous conflicting updates
Unrepeatable Read	Locks control conflicting modifications

Conclusion

2PL ensures **conflict serializability** and provides controlled access to shared database data.

Q6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

Answer:

A **deadlock** occurs when two or more transactions wait indefinitely for resources locked by one another.

Example

T1 locks A

T2 locks B

T1 waits for B

T2 waits for A

Both transactions wait forever.

Deadlock Detection

A DBMS can detect deadlocks using a **Wait-For Graph (WFG)**.

Steps

Step 1: Create one node for every transaction.

T1

T2

T3

Step 2: Draw an edge when one transaction waits for another.

For example:

T1 → T2

means T1 is waiting for T2.

Step 3: Check the graph for a cycle.

Example:

T1 → T2

↑ ↓

└──────┘

A cycle indicates a deadlock.

Step 4: Select a transaction as a victim.

Step 5: Roll back the victim transaction and release its locks.

Deadlock Prevention

1. Wait-Die

This technique uses transaction timestamps.

- Older transaction can wait.
- Younger transaction requesting a resource held by an older transaction is rolled back.

2. Wound-Wait

- Older transaction can force a younger transaction to roll back.
- Younger transaction waits if the lock belongs to an older transaction.

Other prevention methods

- Acquire all required locks before execution.
- Lock resources in a fixed order.
- Use timeout to abort transactions that wait too long.

Conclusion

Deadlock detection identifies cycles in a wait-for graph, while prevention techniques try to ensure that deadlocks do not occur.

Q7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Answer:

Database recovery restores the database to a consistent state after a system failure.

Two important recovery techniques are:

1. Immediate Update
2. Deferred Update

Immediate Update

In immediate update, database changes can be written before the transaction commits.

Therefore, the recovery system may need both:

UNDO + REDO

Example

T1 starts

↓

Update database

↓

Failure occurs

↓

UNDO uncommitted changes

REDO committed changes

Features

- Changes may be written before commit.
- Requires UNDO and REDO.
- Recovery is more complex.
- Provides flexibility in writing data early.

Deferred Update

In deferred update, database changes are not written until the transaction commits.

Recovery mainly uses:

REDO

Example

T1 starts

↓

Changes kept in buffer/log

↓

T1 commits

↓

Changes written to database

If the system crashes before commit, the transaction's changes are simply ignored.

Comparison

Immediate Update	**Deferred Update**
Database may be updated before commit	Database updated after commit
Uses UNDO + REDO	Mainly uses REDO
More complex recovery	Simpler recovery
Uncommitted data may reach disk	Uncommitted data does not reach disk
Suitable for high-performance systems requiring early writes	Suitable when simple recovery is preferred

When preferred?

Immediate Update:

- Large systems
- Systems where changes need to be written early
- Systems using sophisticated logging and recovery

Deferred Update:

- Simpler systems
- Systems where recovery simplicity is important
- Situations where delaying database writes is acceptable

Conclusion

Immediate update provides flexibility but requires more complex recovery, whereas deferred update simplifies recovery by postponing database updates until commit.

Q8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Answer:

Shadow Paging is a database recovery technique that maintains two page tables:

1. Shadow Page Table
2. Current Page Table

The shadow page table represents the stable version of the database.

Working of Shadow Paging

Step 1: Initial State

Both page tables point to the same database pages.

Shadow Table ⟶ Page 1

Current Table ⟶ Page 1

Step 2: Update

When a page needs modification, a new copy is created.

Old Page → Page 1

New Page → Page 2

The current page table points to the new page.

Step 3: Commit

If the transaction completes successfully, the current page table becomes the new valid page table.

Step 4: Failure

If a failure occurs before commit, the DBMS uses the shadow page table.

Therefore, the old database state can be recovered.

Advantages

1. Simple Recovery

No complicated UNDO operation is required.

2. Fast Rollback

The system can return to the old page table.

3. No Need for Complex Log Processing

Unlike log-based recovery, recovery can be performed using page-table information.

4. Original Data is Preserved

Modified pages are written separately.

Limitations

1. Extra Storage

Copies of modified pages require additional space.

2. Fragmentation

Frequent page copying can result in fragmented storage.

3. Page Table Overhead

Managing page tables can require significant memory.

4. Poor for Large Databases

Managing many pages can become expensive.

Shadow Paging vs Log-Based Recovery

Shadow Paging	**Log-Based Recovery**
Uses shadow and current page tables	Uses log records
Recovery is simple	Recovery can be complex
Extra page-table management	Requires log management
Can cause fragmentation	Generally better suited for large databases
Limited support for concurrent transactions	Better support for concurrency

Conclusion

Shadow paging is simple and provides fast rollback, but log-based recovery is generally more flexible and scalable for large database systems.

Q9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

Answer:

A schedule is the order in which operations of multiple transactions are executed.

A schedule is serializable if its result is equivalent to the result of executing the transactions serially.

Serial Schedule

Transactions execute one after another.

Example:

T1 → T2

T1 completely finishes before T2 starts.

Concurrent Schedule

Operations of transactions are interleaved.

Example:

T1: Read A

T2: Read B

T1: Write A

T2: Write B

A concurrent schedule is useful for performance but must maintain correctness.

Conflict Serializability

Two operations conflict when:

1. They belong to different transactions.
2. They access the same data item.
3. At least one operation is a write.

Conflicting pairs are:

Read – Write

Write – Read

Write – Write

Precedence Graph

A precedence graph is used to test conflict serializability.

Steps

Step 1: Create a node for each transaction.

T1, T2, T3

Step 2: Identify conflicting operations.

Step 3: If an operation of T1 occurs before a conflicting operation of T2, create:

T1 → T2

Step 4: Check for cycles.

Rule

No cycle → Conflict Serializable

Cycle → Not Conflict Serializable

Example

Suppose:

T1 → T2

T2 → T3

T1 → T3

There is no cycle.

Therefore, the schedule is **conflict serializable**.

But if:

T1 → T2

T2 → T1

there is a cycle.

Therefore, the schedule is not conflict serializable.

Conclusion

The precedence graph provides a simple way to determine whether a concurrent schedule is conflict serializable.

Q10. Describe Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

Answer:

Two-Phase Locking (2PL) is a concurrency control protocol used to ensure conflict serializability.

It has two phases:

1. Growing Phase

The transaction can acquire locks but cannot release locks.

2. Shrinking Phase

The transaction can release locks but cannot acquire new locks.

1. Basic 2PL

In Basic 2PL:

- Locks are acquired during the growing phase.
- Locks are released during the shrinking phase.
- Once a transaction releases a lock, it cannot acquire another lock.

Advantage

It guarantees conflict serializability.

Limitation

It can allow cascading rollbacks.

2. Strict 2PL

In Strict 2PL:

- All **exclusive locks (X-locks)** are held until commit or abort.
- Shared locks may be released earlier depending on the implementation.

Example:

T1 obtains X-lock

↓

Updates data

↓

COMMIT

↓

X-lock released

Advantages

- Prevents cascading rollback.
- Simplifies recovery.
- Ensures conflict serializability.

3. Rigorous 2PL

In Rigorous 2PL:

Both shared and exclusive locks are held until the transaction commits or aborts.

S-lock → held until COMMIT

X-lock → held until COMMIT

It provides stronger isolation than Strict 2PL.

Comparison

Feature	Basic 2PL	Strict 2PL	Rigorous 2PL
Conflict serializable	Yes	Yes	Yes
X-lock held until commit	No	Yes	Yes
S-lock held until commit	No	Usually no	Yes
Cascading rollback	Possible	Prevented	Prevented
Recovery	More difficult	Easier	Easier
Locking strength	Normal	Strong	Strongest

Which is commonly used?

Strict 2PL is commonly used because it provides a good balance between:

- Concurrency
- Data consistency
- Recovery
- Simplicity

It prevents transactions from reading/writing data that may later be rolled back and makes database recovery easier.

Conclusion

Basic 2PL provides serializability, Strict 2PL additionally improves recoverability, and Rigorous 2PL provides the strongest lock retention. Strict 2PL is preferred in many practical DBMS applications because it provides strong correctness while maintaining reasonable concurrency.